CSU4405 Computer Graphics Final Project

"Toward a Futuristic Emerald Isle"

-Stephen Komolafe (21336975)

1. Application Introduction

The objective of this final project, themed "Towards a Futuristic Emerald Isle," was to develop a shader-based OpenGL application that showcases the computer graphics techniques learnt throughout the module by creating an immersive 3D environment. Inspired by the theme, I chose to depict a futuristic version of North Dublin's city centre, featuring flying cars, advanced buildings, and intricate terrain. Beyond the foundational features covered in labs 1-4—geometry rendering, texture mapping, lighting, and animation—this project also incorporated additional core elements, including mouse and keyboard camera controls, a functional skybox, successful importing and rendering of custom Blender models, looping camera boundaries for an infinite space effect, toggleable fullscreen mode, and a real-time FPS counter for performance monitoring. As a solo project, my goal was to demonstrate my understanding of OpenGL programming and my ability to create custom Blender models to bring the project to life.

2. Progress Report

[1.] The project began with a blank 3D scene featuring keyboard and mouse-controlled camera movement. I added a simple plane to better visualize the camera's three-dimensional movement within the scene.

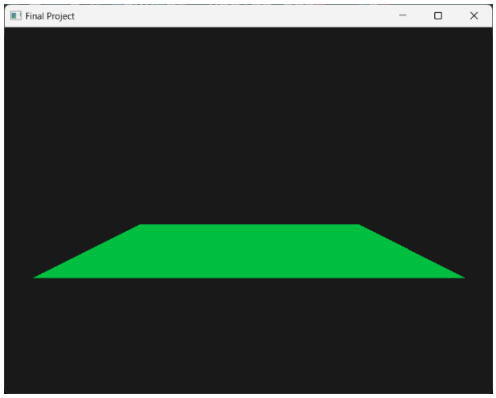

Fig 1. Plane in 3d space

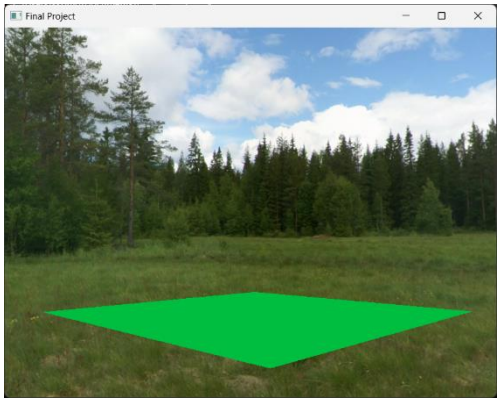

Fig 2. Placeholder Meadow Skybox

[2.] A skybox was implemented along with a vertex and fragment shader. Placeholder textures were used initially, with the intention of replacing them with more thematically fitting textures later in development, while also using them to learn how to properly assign textures to a skybox.

[3.] Functionality was added to integrate Blender models into the application by exporting them as wavefront (.obj) files. A default Blender cube was used initially to test the integration, with plans to replace it with more detailed models later on.

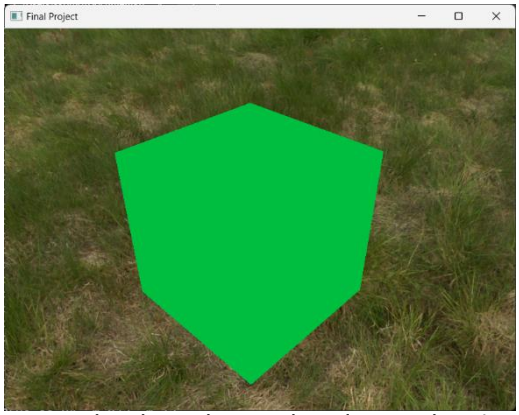

Fig 3. Blender cube rendered in application
(Green since it shares same shaders as plane)

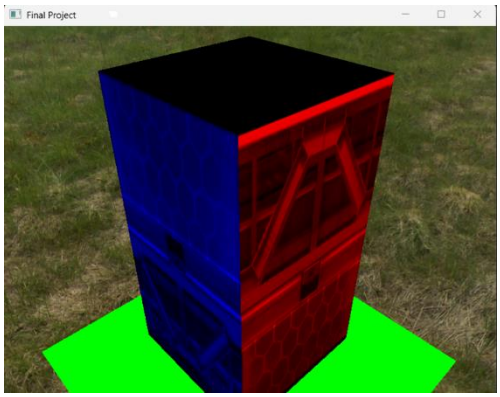

Fig 4. Prototype building with buggy shaders
(eventually fixed but used on new buildings)

[4.] Dedicated vertex and fragment shaders were created for the models to properly handle the loading of textures from each model's associated material file (.mtl). A cuboid, textured via UV mapping, was used to test the correct operation of these shaders.

[5.] Three types of abstract buildings were modelled and textured in Blender for use in the scene. To represent North Dublin's city centre, a futuristic version of the Spire was also created.

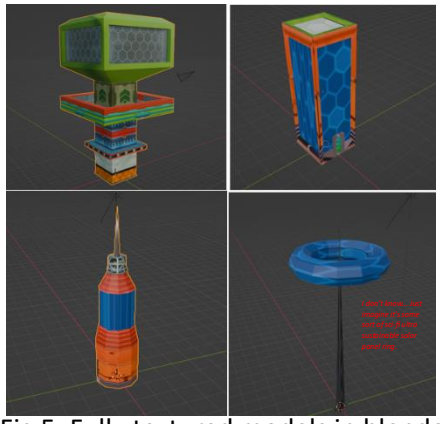

Fig 5. Fully textured models in blender

[6.] The coordinates of each building were manually plotted and integrated into the application. The plane size was increased to serve as the ground for the scene, and the application's title was updated to reflect the project's theme.

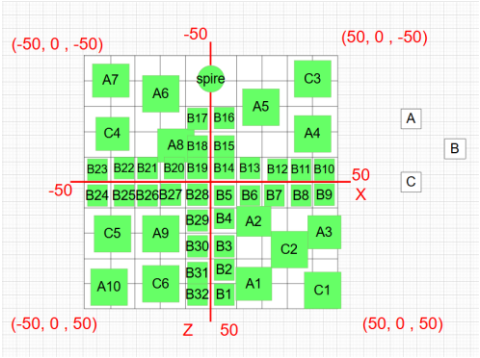

Fig 6a). Coordinates of each buildings position
(green if integrated/white if not)

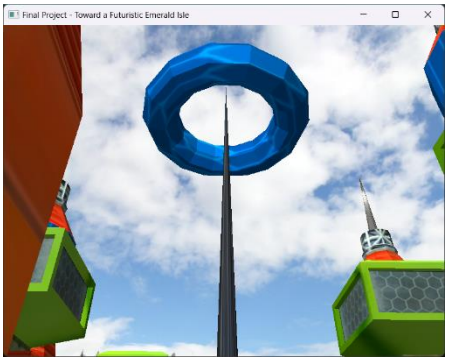

Fig 6b). Integrated Spire at runtime

Fig 6c). Integrated type A, B, and C buildings at runtime

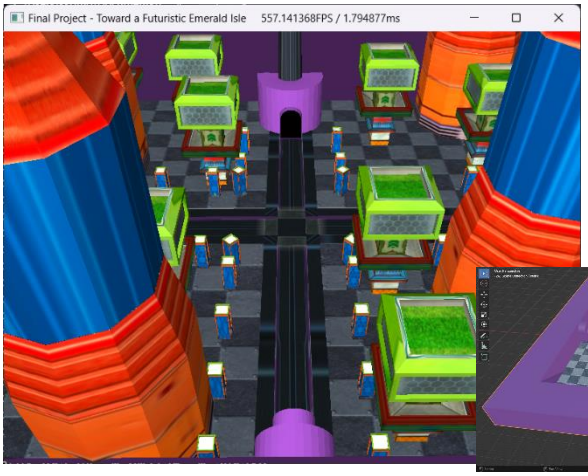

Fig 7a). Ground model at runtime

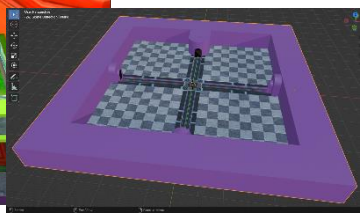

Fig 7b). Ground model in blender

[7.] A more detailed ground model was created to replace the simple plane, featuring a junction road, pavements, and tunnels (for the looping camera boundary implementation). Real-time FPS/ms counters were also added next to the application's title (precision changed to 2d.p. and Vsync enabled later to prevent the fps from reaching into the 100s).

[8.] To introduce animation, a flying car was modelled and textured. Multiple cars were rendered within the scene, each assigned its own flight path. The placeholder skybox was replaced with a more fitting one, and a toggleable fullscreen mode was implemented via the 'F' key.

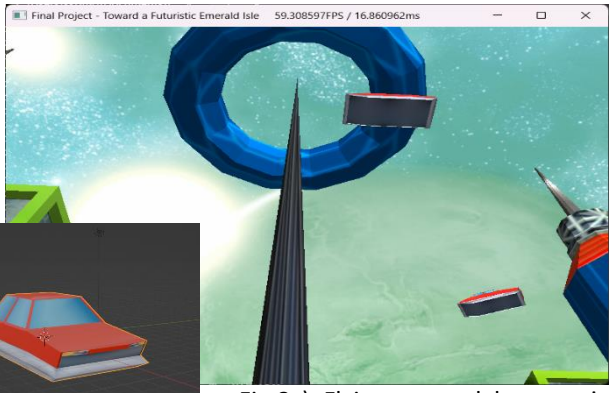

Fig 8a). Flying car models at runtime +Interstellar skybox

Fig 8b). Flying car model in blender

[9.] A looping camera boundary was created to simulate infinite space and prevent the camera from exiting the scene. The camera now wraps around to the opposite side when it reaches the boundary (this functionality cannot be captured as an image during runtime, but here's the function used in camera.cpp).

```
49  void Camera::Loop() {
50      if (Position.x > LoopBounds.x)
51          Position.x = -LoopBounds.x;
52      else if (Position.x < -LoopBounds.x)
53          Position.x = LoopBounds.x;
54
55      if (Position.y > LoopBounds.y)
56          Position.y = -LoopBounds.y;
57      else if (Position.y < -LoopBounds.y)
58          Position.y = LoopBounds.y;
59
60      if (Position.z > LoopBounds.z)
61          Position.z = -LoopBounds.z;
62      else if (Position.z < -LoopBounds.z)
63          Position.z = LoopBounds.z;
64  }
```

Fig 9). Camera.cpp Loop function

3. Discussion - Application Quality & Robustness

To enhance the application's robustness, I focused on modularising its features as much as possible. Core components such as the ground plane (while it was still in use), camera, model, mesh, shader, and skybox were separated into dedicated C++ and header files, which were then called in the main code. This modular design significantly streamlined maintenance and debugging of the code, making the application more manageable. Additionally, extra features such as fullscreen mode, looping camera boundaries, and varied animation paths for the flying cars, though not essential, added to the overall quality and polish of the project. Despite its complexity, the application remains stable, with efficient rendering loops that allow for real-time logging of textures and vertices for successfully

loaded models. Performance optimisation was further improved by implementing features like a VSync toggle, ensuring the application runs smoothly without erratic frame rates.

4. Discussion - Limitations & Potential Future Work

Although the project timeline was generous, I feel there are still many features and optimisations I would have liked to implement. During the initial implementation of model importing in the main.cpp file, each building was rendered manually—that is, calling the same building model multiple times and making separate draw calls (modelA1, modelA2, modelA3, …). This resulted in redundant code that unnecessarily bloated the file and made navigation more cumbersome. While this was eventually resolved by using for loops to iterate through vectors containing building positions and rotations, the approach may still appear slightly repetitive to some. Towards the latter part of the project, I attempted to incorporate instancing to render multiple instances of each building type simultaneously but had to abandon this as it caused errors of which there wouldn't be enough time to debug with the deadline drawing nearer. On the modelling side, learning Blender was an enjoyable experience, and with more time, I would have been keen to create and texture additional types of buildings, cars, and monuments to further enrich and populate the city.

5. Acknowledgement

I would like to acknowledge the following open data and source code that greatly assisted in the creation of this computer graphics project: GLFW (1), Glad (2), glm (3), Open Asset Import Library (4), and stb single-file public domain libraries for C/C++ (5). I also appreciate the guidance from "OpenGL Tutorials" by Victor Gordan (6) for project setup and "Low Poly Building (Blender Tutorial)" by Ryan King Art (7) for learning Blender. Textures were sourced from various resources, including building textures (8), ground textures [Futuristic Road Design by Joshua Tinoco (9); Checkered floor design (10); Hexagonal floor design (8); Underside texture (11)], and car textures (12). The placeholder skybox, "Meadow" by Emil Persson (13), and final skybox, "Interstellar" by Jockum Skoglund (14), were also utilized. Lastly, specific queries, debugging support, and optimisation advice were provided by ChatGPT (15).

1. https://www.glfw.org/
2. https://glad.dav1d.de/
3. https://github.com/g-truc/glm
4. https://github.com/assimp
5. https://github.com/nothings/stb
6. https://www.youtube.com/@VictorGordan
7. https://www.youtube.com/@RyanKingArt
8. https://www.textures-resource.com/gamecube/sonicheroes/texture/2798/?source=genre
9. https://www.artstation.com/josh_tinoco
10. https://www.textures-resource.com/gamecube/supermonkeyball2/texture/23865/?source=genre
11. https://www.pinterest.com/pin/986640230849118733/
12. https://en.wikipedia.org/wiki/Lightning_McQueen
13. https://opengameart.org/content/field-skyboxes
14. https://opengameart.org/content/interstellar-skybox
15. https://chatgpt.com/